

What the 010 Trigenic Model Is Measured Against

Additive nulls, what each model actually sees, and whether the comparison is fair

Michael Volk

September 1, 2026

Abstract

A model that claims to learn genetic interaction has to beat a null that cannot represent interaction. Fitting that null on experiment 010 puts a per-gene additive ridge at 0.400 test Pearson against the cell graph transformer’s 0.438 to 0.455, and puts a model that sees only which recurring gene pair a triple contains, ignoring the third gene entirely, at 0.390. Reading the model code establishes what the comparison is between: the transformer’s encoder never sees the strain, the nine interaction graphs never enter the forward pass, and the strain-dependent computation is a set function over the three perturbed gene identities, which is the same input and the same hypothesis-class shape the nulls have. Rebuilding the forward pass from that reading reproduces the recorded metrics to the fourth decimal and yields per-record predictions. Those show the transformer is not a relabeled additive model, correlating with it at 0.73 to 0.76 rather than the 0.999 reported for MULTI-evolve, and that its gain over the null is +0.04 to +0.055 Pearson with bootstrap intervals excluding zero. They also show two training runs of the architecture agree with each other at 0.79 to 0.82, so the non-additive content is largely not reproducible across seeds. Separately, the transformer’s predictions over the unmeasured design space do not agree between inference runs of one checkpoint, which is a defect in the runs that produced the panel selection rather than a fact about additivity.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Provenance: ▲ not in the library mirror; verify at the linked source ● from a review’s summary table, not the primary paper

Contents

1	The question ✗	3
2	What is being predicted ✗	3
2.1	The label ✗	3
2.2	The build ✗	3
2.3	The screen design, which is the confounder ✗	4
3	What each model actually sees ✗	4
3.1	The transformer’s per-sample input ✗	4
3.2	The encoder does not depend on the strain ✗	4
3.3	The nine graphs do not enter the prediction ✗	5
3.4	The transformer is a set function on three genes ✗	5
4	The models ✗	5
4.1	Notation and the interaction operator ✗	5
4.2	B0, the train mean ✗	6
4.3	B1, the additive per-gene ridge ✗	6
4.4	B2, additive plus recurring gene pairs ✗	6
4.5	B3 and B4, the screen-structure baselines ✗	6
4.6	B5, nonlinearity on the same features ✗	6
4.7	Summary of capacity ✗	6
5	Is the comparison fair ✗	7
5.1	Which split the reported numbers come from ✗	7
5.2	What the transformer metrics are ✗	7
5.3	What each model sees ✗	7
5.4	Where the comparison is not clean ✗	7
5.5	Model size ✗	8
6	Held-out results ✗	8
6.1	Reading the ladder ✗	8
6.2	Verifying the architecture reading ✗	10
6.3	Record-by-record agreement ✗	10
7	The design space, and a defect found there ✗	12
7.1	The comparison as run ✗	12

7.2	One checkpoint disagreeing with itself	12
7.3	What is and is not established	13
7.4	Seed instability in the selection tail, separately	13
8	What this establishes, and what it does not	13
8.1	Established	13
8.2	Not established	13
8.3	Next	14

1 The question ✗

Tran et al. introduced MULTI-evolve in *Science* in 2026 (10.1126/science.aea1820), a protein engineering framework whose fully connected networks were said to “learn the epistatic landscape” from single and double mutants and to identify synergistic combinations at higher order. Visani, Verma and DeWitt replied in a preprint, *Additive baselines furnish no evidence for epistasis learning by MULTI-evolve* (bioRxiv 2026.04.23.719915), which does one thing: it fits the null that the original paper never fit.

An **additive model** over mutations predicts a phenotype as a sum of per-mutation terms. By construction it assigns exactly zero to every interaction. It is therefore the natural null for any claim that a model has learned interaction, in the same way that an intercept-only model is the null for a claim that a model has learned anything.

Their result is that the network’s predictions across the combinatorial variant space correlate with a ridge-regularized additive model’s at $r > 0.999$ for all three engineering campaigns, that the variants selected for construction are ranked nearly identically by both, and that on held-out test data the network does not significantly outperform either the additive model or a zero-hidden-layer version of its own architecture. The engineering result is real; the attribution to epistasis learning is not supported.

The analogous question for experiment 010 is whether the equivariant cell graph transformer’s held-out performance on the trigenic interaction score survives the same test. Traditional machine-learning baselines exist in this repository for the mixed digenic and trigenic set under `experiments/002-dmi-tmi` and for fitness under `experiments/smf-dmf-tmf-001`. None had been fit on the 010 build, which is triples only.

One difference in the setup, and it cuts against the null. In the protein setting the quantity predicted is the multimutant phenotype itself, which per-mutation effects explain directly. In 010 the quantity predicted is already a residual. The trigenic interaction score subtracts the expectation built from single and double mutant fitness before the model ever sees it, so the additive part of fitness has been removed by construction. An additive-in-single-gene-effects model is therefore a harder null here than it is there, and its performance is more surprising, not less.

That argument has one loophole, and quantifying it turns out to matter more than anything else in this document. It is taken up in Section 6 as baseline B4.

2 What is being predicted ✗

2.1 The label ✗

Write f_i for the fitness of the single mutant Δi , f_{ij} for the double, f_{ijk} for the triple, each relative to wild type. The **digenic interaction score** is the departure of the double from the multiplicative expectation,

$$\varepsilon_{ij} = f_{ij} - f_i f_j, \quad (1)$$

and the **trigenic interaction score** is the departure of the triple from the expectation built out of the singles *and* the doubles,

$$\tau_{ijk} = f_{ijk} - f_i f_j f_k - f_i \varepsilon_{jk} - f_j \varepsilon_{ik} - f_k \varepsilon_{ij}. \quad (2)$$

▲ [external: Kuzmin et al. 2018, 10.1126/science.aao1729; the published definition of the released adjusted score column, not re-derived here]

Two properties of Eq. 2 matter for everything that follows. It is a *residual*: the additive-on-the-log-scale, multiplicative-on-the-linear-scale part of fitness is subtracted before the number is reported. And it is a *third-order* residual: the pairwise terms are subtracted too, so τ is by construction the part of the triple mutant phenotype that neither single nor double mutant measurements predict.

The 010 loader consumes the column **Adjusted genetic interaction score (epsilon or tau)** from the Kuzmin releases, which for a trigenic record is τ , and stores it under the label key `gene_interaction`.

2.2 The build ✗

The build is triples only, so there are no singles or doubles in it. The MULTI-evolve experiment of training on low order and extrapolating to high order is therefore not available here, and the question is the narrower one: given a training set of triples, how much of the held-out τ is recoverable without representing interaction.

Records	376,732
Perturbation count	3 for every record
Distinct perturbed genes	4,352
Sources	TmiKuzmin2018Dataset, TmiKuzmin2020Dataset
Label	gene_interaction (τ)
Split, seed 42	train 301,386 / val 37,673 / test 37,673
Train label SD	0.063186
Val label SD	0.062914
Test label SD	0.064187

Table 1: The 010 build. Every baseline and the transformer use these records, this split and this label. Read from `001-small-build/processed/label_df.parquet`, `is_any_perturbed_gene_index.json` and `data_module_cache/index_seed_42.json`.

2.3 The screen design, which is the confounder $\times$

Kuzmin’s trigenic screen crosses a **query** double mutant against an array of single mutants, so a triple is a query pair plus one array gene, and the same query pair recurs across hundreds or thousands of triples. Counting unordered gene pairs in the training split finds 593,978 distinct pairs, of which only 420 occur five or more times, the most frequent occurring 3,308 times. That concentration is the query-pair structure.

The 010 split is random over records, so a query pair that appears in training also appears in validation and test. Any model that can represent a per-pair offset can absorb whatever is common to a query batch, and none of that is trigenic biology. Baseline B4 in Section 4 is built to measure exactly this quantity.

3 What each model actually sees $\times$

The comparison is only meaningful if the models are given the same information. Reading the 010 code settles that, and the answer is stronger than expected: every model in this document, the transformer included, receives exactly one thing per record, the set of three perturbed genes.

3.1 The transformer’s per-sample input $\times$

The 010 dataset is built with the `Perturbation` graph processor. That processor writes the perturbed gene names, their indices, a boolean mask and the label into each sample, and nothing else (`graph_processor.py`, lines 1984 to 2058). There is no per-gene feature matrix: with an empty `node_embeddings` list in the config, the gene feature tensor of the cell graph has shape 6607×0 (`cell_data.py`, line 53). Gene identity is carried by a free lookup table of 6,607 rows and 180 columns (`equivariant_cell_graph_transformer.py`, line 1989), initialized randomly and fit by gradient descent.

No sequence, expression, ontology or other precomputed representation enters. The model is given gene identities, the same as a one-hot design matrix, and learns its own 180-dimensional vector for each.

3.2 The encoder does not depend on the strain $\times$

The forward pass builds its input as `gene_embedding(torch.arange(N))`, prepends a CLS token, and unsqueezes to shape $[1, N+1, 180]$. That leading dimension is 1, not the batch size. The eight transformer layers then run on this single tensor, which is assembled entirely from parameters and contains nothing from `batch`. Writing E for the embedding table and T for the eight-layer encoder,

$$Z = T(E) \in \mathbb{R}^{N \times 180}, \quad h_{\text{CLS}} \in \mathbb{R}^{180}, \quad (3)$$

and Z and h_{CLS} are the same for every record in the dataset. The repository states this itself, in the docstring of `PerturbationGraphPropagation`: “The encoder runs at batch 1 on the WILDTYPE graph, so `H_genes` and `h_CLS` are identical for every strain; the only strain-dependent step is `EquivariantPerturbationTransform`.”

So the eight encoder layers, which hold 3,129,120 of the model’s 4,774,861 parameters, perform no per-sample computation. They map one free parameter block to another. Whatever role they play, it is not to process the strain.

3.3 The nine graphs do not enter the prediction ✗

The model has three places adjacency could act. In the 010 configuration all three are off. The attention-mask configuration is not passed, so the head mask is `None` and every layer receives no mask. The perturbation-propagation configuration is not passed either, so that block is skipped. The attention itself is ordinary softmax self-attention with no adjacency bias term.

The nine adjacency matrices are used in one place only, a Kullback-Leibler penalty between each normalized adjacency and one attention head of layer 1, which is added to the *loss*. Setting that penalty’s weight to zero therefore cannot change any prediction. Section 6 reports the measurement rather than resting on the reading.

Two further facts about the 010-era code bear on how much the penalty could have shaped Z even indirectly. The configuration names `physical` and `regulatory`, while the cell graph emits `physical_interaction` and `regulatory_interaction`, and the code of that period skipped any name it did not find, so two of the nine heads were silently unregularized. And the penalty weight was applied twice from the same configuration key, giving an effective weight of 10^{-6} rather than 10^{-3} . Both were fixed in July 2026, after these runs.

3.4 The transformer is a set function on three genes ✗

What remains after Eq. 3 is the strain-dependent part. For a record with perturbed set S , $|S| = 3$, the perturbation transform performs cross-attention in which all N genes query the $|S|$ rows $\{Z_p : p \in S\}$, followed by a residual, a layer norm and a feedforward block, giving $H_i^{\text{pert}} = g(Z_i + c_S)$ where c_S is the attended context. The head then pools over the perturbed genes and reads out,

$$z_S = \frac{1}{|S|} \sum_{i \in S} H_i^{\text{pert}}, \quad \hat{y} = \text{MLP}([h_{\text{CLS}} \parallel z_S]), \quad (4)$$

with the MLP being $\text{Linear}(360, 180) \rightarrow \text{ReLU} \rightarrow \text{Linear}(180, 1)$.

Since h_{CLS} is constant, Eq. 4 is a function of z_S alone, and z_S is a permutation-invariant pooling of quantities derived from $\{Z_p : p \in S\}$. The 010 transformer is therefore a set function on the three perturbed gene identities, built on a free per-gene vector. That is the same functional shape as the additive nulls, which are set functions on the same three identities built on a free per-gene scalar. The models differ in how expressive the set function is, not in what they are allowed to see.

This is what makes the comparison in Section 6 fair, and it also corrects a claim made in an earlier version of the working note, that the gap between the nonlinear baseline and the transformer was attributable to the nine interaction graphs. The graphs are not in the forward pass, so that gap cannot be attributed to them.

4 The models ✗

4.1 Notation and the interaction operator ✗

Let G be the 4,352 genes appearing in the build and let record n have perturbed set $S_n \subset G$ with $|S_n| = 3$. Encode it as $x_n \in \{0, 1\}^{|G|}$ with $x_{n,g} = 1$ exactly when $g \in S_n$, so that $\|x_n\|_1 = 3$ for every record. Write y_n for the trigenic interaction score of Eq. 2.

A model is a set function $f : 2^G \rightarrow \mathbb{R}$. Its capacity to represent interaction is measured by finite differences. For distinct genes g, h, k and any S disjoint from them, define

$$\Delta_g f(S) = f(S \cup \{g\}) - f(S), \quad (5)$$

$$\Delta_{gh} f(S) = f(S \cup \{g, h\}) - f(S \cup \{g\}) - f(S \cup \{h\}) + f(S), \quad (6)$$

$$\Delta_{ghk} f(S) = \sum_{U \subseteq \{g, h, k\}} (-1)^{3-|U|} f(S \cup U). \quad (7)$$

$\Delta_{gh} f \equiv 0$ says the model cannot express any pairwise interaction; $\Delta_{ghk} f \equiv 0$ says it cannot express any three-way interaction. These are the properties that make a model a null, and they are structural, holding for every parameter setting rather than for a fitted one.

4.2 B0, the train mean ✗

$\hat{y} = \bar{y}_{\text{train}}$. Every finite difference vanishes. Its mean squared error is the label variance, and it fixes the zero point against which all reductions are read.

4.3 B1, the additive per-gene ridge ✗

$$\hat{y}(S) = \beta_0 + \sum_{g \in S} \beta_g, \quad \hat{\beta} = \arg \min_{\beta} \sum_{n \in \text{train}} (y_n - \beta_0 - x_n^\top \beta)^2 + \alpha \|\beta\|_2^2, \quad (8)$$

with β_0 unpenalized and set to $\bar{y}_{\text{train}}$. Substituting Eq. 8 into Eq. 6 gives $\Delta_{gh}\hat{y} = 0$ and hence $\Delta_{ghk}\hat{y} = 0$ identically. B1 is the direct analog of the additive null in the DeWitt preprint.

The design is well conditioned despite having 4,352 coefficients: with three ones per row and 301,386 training rows there are 904,158 gene observations, a median of 9 per gene in the training split with a maximum of 3,430. The ridge penalty is chosen on validation over $\alpha \in \{10^{-2}, 10^{-1}, 1, 3, 10, 30, 100, 300, 1000\}$ and the selected value is then reported on test.

4.4 B2, additive plus recurring gene pairs ✗

Let P be the set of unordered gene pairs occurring at least five times in the training split, $|P| = 420$. Adding one coefficient per such pair,

$$\hat{y}(S) = \beta_0 + \sum_{g \in S} \beta_g + \sum_{\{g,h\} \subseteq S, \{g,h\} \in P} \gamma_{gh}, \quad (9)$$

gives $\Delta_{gh}\hat{y} = \gamma_{gh}$ for a pair in P and zero otherwise. B2 therefore has second-order capacity on observed pairs. It still has *no* third-order capacity: $\Delta_{ghk}\hat{y} = 0$ identically. Since the label is itself a third-order residual, B2 remains a null with respect to the quantity being predicted.

4.5 B3 and B4, the screen-structure baselines ✗

B4 drops the per-gene terms from Eq. 9 and keeps only the pair terms,

$$\hat{y}(S) = \beta_0 + \sum_{\{g,h\} \subseteq S, \{g,h\} \in P} \gamma_{gh}. \quad (10)$$

Because the recurring pairs are the Kuzmin query doubles, B4 is close to a per-query-batch offset. For a triple made of a query pair and an array gene, the array gene contributes nothing at all to Eq. 10. Whatever B4 achieves is therefore an upper bound on how much of the metric is explained by knowing which screen a record came from.

B3 is the unregularized empirical counterpart: predict the mean training label over triples sharing each recurring pair, backing off to the mean over triples containing each gene, and then to the global mean.

4.6 B5, nonlinearity on the same features ✗

Give each gene a free vector $e_g \in \mathbb{R}^{128}$, sum over the perturbed set and read out with a two-hidden-layer network ϕ ,

$$\hat{y}(S) = \phi \left(\sum_{g \in S} e_g \right), \quad \phi : \mathbb{R}^{128} \rightarrow \mathbb{R}. \quad (11)$$

This is permutation invariant and uses precisely B1's feature space, the identity of the three genes and nothing more. Because ϕ is nonlinear, $\Delta_{gh}\hat{y}$ and $\Delta_{ghk}\hat{y}$ are not identically zero, so B5 is not a null. It is the smallest departure from B1 that has interaction capacity, and it isolates what capacity alone buys.

Comparing Eq. 11 with Eq. 4 shows they are the same construction. Both give every gene a free vector, pool over the three perturbed genes in a permutation-invariant way, and pass the pooled vector through a small network. The transformer's pooling is richer, since the context c_S is produced by attention over the perturbed set rather than by a plain sum, and its per-gene vectors are $Z = T(E)$ rather than a raw table. Trained with three seeds, B5 reports a spread rather than a single number.

4.7 Summary of capacity ✗

Model	Per-gene representation	Set aggregation	$\Delta_{gh} \neq 0$	$\Delta_{ghk} \neq 0$
B0	none	none	no	no
B4	none	recurring pair indicators	yes	no
B3	scalar mean	pair mean, backing off	yes	no
B1	free scalar	sum	no	no
B2	free scalar	sum plus pair terms	yes	no
B5	free 128-vector	sum, then MLP	yes	yes
CGT	free 180-vector via $T(E)$	attention over S , mean, MLP	yes	yes

Table 2: What each model can represent. The last two columns are structural properties of the hypothesis class, not fitted results. Only B5 and the cell graph transformer can express a three-way interaction, which is what the label is.

5 Is the comparison fair ✗

Six things have to match before a baseline number can sit in the same table as a transformer number. Five do. One does not, and it favors the transformer by a small amount.

5.1 Which split the reported numbers come from ✗

Both validation and test numbers are available for the transformer, and the table in Section 6 reports **test**. The distinction matters because the three transformer checkpoints are *best-Pearson* checkpoints, selected on validation across roughly 25 to 64 validation epochs. A validation Pearson chosen that way is the maximum of many draws and is upward-biased as an estimate of capability; its test Pearson is not, because test was not consulted during selection.

The baselines follow the same discipline. B1, B2 and B4 select the ridge penalty on validation and report test. B5 early-stops on validation and reports test. So the test column compares like with like, and the validation column is biased in the same direction for every row.

5.2 What the transformer metrics are ✗

The metrics come from the three checkpoints’ re-evaluation runs under `$DATA_ROOT/wandb-experiments`, read from `files/wandb-summary.json` rather than retyped. Two families of key exist and they are not interchangeable.

- `val/gene_interaction/{MSE,RMSE,Pearson}` and the `test/` equivalents are torchmetrics objects updated once per batch and computed at epoch end, so they cover the *full* 37,673 records of each split. They are computed after the inverse transform, in raw τ units.
- `val_sample/Spearman_target_0` and its test counterpart come from a separate path that collects samples for plotting and is capped by `plot_sample_ceiling`. In the evaluation configuration that ceiling is 10^6 , above the split size, so no truncation fires and the Spearman is also over the full split. In the *training* configuration the same ceiling is 10,000, so a Spearman quoted from a training run would be over a random 10,000 of 37,673. The numbers here are from the evaluation runs.
- `val/transformed/gene_interaction/*` are the same metrics in standardized units and are not used here.

The baselines compute Pearson, Spearman and mean squared error over the same full splits in raw τ units, so the units and the denominators agree.

5.3 What each model sees ✗

Established in Section 3: the perturbed gene set, and nothing else, for every model in the comparison. No baseline is handicapped by being denied a feature the transformer had, and the transformer is not credited with information it never received.

5.4 Where the comparison is not clean ✗

Label normalization leaks. The transformer trains on standardized labels, and the standardization statistics are fit on *all* 376,732 records rather than on the training split. The transform is constructed from `dataset.label_df`, which

iterates every record in the LMDB with no split filter, and it is constructed before the data module splits the data. The logged constants confirm it: mean -0.008024324 and standard deviation 0.063263549 match the all-record statistics to nine digits and do not match the train-only values -0.007688739 and 0.063186255 .

The leak is two global scalars, and the train and all-record values differ by about 3×10^{-4} in the mean and 10^{-4} in the standard deviation, so the effect is small. It is nonetheless real, it runs in the transformer’s favor, and the baselines do not have it: they use the training mean only. No correction is applied here, and no attempt is made to estimate the size of the advantage.

The loss functions differ, deliberately. The baselines minimize squared error. The transformer minimizes

$$\mathcal{L} = \lambda_p \mathcal{L}_{\text{MSE}} + \lambda_d S_\epsilon\left(\frac{1}{B} \sum_b \delta_{\hat{y}_b}, \frac{1}{B} \sum_b \delta_{y_b}\right) + \lambda_g \mathcal{L}_{\text{graph}}, \quad (12)$$

with $\lambda_p = 1$, $\lambda_d = 0.1$, $\lambda_g = 1$. The middle term is a debiased Sinkhorn divergence with $p = 2$ and blur 0.05 between the empirical measure of the batch’s predictions and that of the batch’s targets, computed unpaired, which shapes the marginal distribution of the predictions rather than their per-record accuracy. The third term is the graph penalty of Section 3.3.

This is a difference in training objective, not in the evaluation, and it is part of what the transformer is. It is noted because a distributional term can move Pearson and mean squared error in opposite directions, so the two metrics should be read together rather than either alone.

Paired residuals are not available from the recorded runs. The original evaluation runs wrote per-record prediction files to the cluster checkout, and those files are not on this machine. Recomputing them through the stock evaluation script fails, because the 010 LMDB predates the perturbation-ontology refactor: every record carries `perturbation_type: "deletion"` where the current pydantic schema admits only `sga_kanmx_deletion` or `sga_natmx_deletion`, which is 204 validation errors on the first batch. Section 6 reports how far this was worked around.

5.5 Model size ✗

Component	Parameters	Role
Gene embedding table	1,189,260	the only place gene identity lives
Encoder, 8 layers	3,129,120	strain-independent, see Sec. 3.2
Perturbation transform	391,140	the strain-dependent step
Readout head	65,161	pooling and MLP
CLS token	180	constant
Total	4,774,861	
B1 additive ridge	4,353	one scalar per gene, plus intercept
B5 embedding MLP	623,745	128 per gene, plus a two-layer network

Table 3: Parameter budget. Two thirds of the transformer sits in the encoder, which does no per-sample work, and a quarter in the embedding table. The strain-dependent computation is 456,301 parameters, under a tenth of the model.

6 Held-out results ✗

6.1 Reading the ladder ✗

The additive null is close, but the transformer does beat it. B1 reaches 0.400 against the transformer’s 0.438 to 0.455, so a model that cannot represent any interaction recovers 88 to 91 percent of the correlation. In explained variance the gap is $r^2 = 0.160$ against 0.207 for the best checkpoint, an increment of 0.047 of label variance. In error, the best checkpoint reduces mean squared error 19.6 percent below B0 and B1 already delivers 16.1 percent of that. This is not the DeWitt result, where the network collapsed onto the additive model at $r > 0.999$. The margin here is real. It is also small.

Seed spread is a large fraction of that margin. The three transformer runs span 0.438 to 0.455, a standard deviation of 0.0088. The three B5 seeds span 0.425 to 0.427, a standard deviation of 0.0011. So the transformer’s run-to-run spread is eight times the nonlinear baseline’s and is roughly a fifth of its own margin over B1. A single checkpoint’s number is not the capability.

Model	Interaction capacity	Pearson	Spearman	MSE
B0 train mean	none	0.000	0.000	0.004124
B4 query pair only	pairwise, 420 pairs	0.390	0.362	0.003493
B3 hierarchical mean	pairwise, empirical	0.390	0.362	0.003493
B1 additive per-gene ridge	none	0.400	0.406	0.003460
B2 additive plus pair ridge	pairwise, 420 pairs	0.406	0.412	0.003442
B5 embedding MLP, seed 0	full	0.425	0.417	0.003382
B5 embedding MLP, seed 1	full	0.427	0.421	0.003395
B5 embedding MLP, seed 2	full	0.425	0.419	0.003381
CGT M01 lzs9pcj3	full	0.443	0.426	0.003400
CGT M02 yv4r30bi	full	0.438	0.424	0.003439
CGT M03 c7671wgj	full	0.455	0.426	0.003315

Table 4: Test split, 37,673 records, raw τ units. The interaction-capacity column repeats Table 2: only the last four rows can express a three-way interaction, which is what the label is. Ridge penalties are selected on validation; B5 early-stops on validation; the transformer checkpoints are selected on validation. Test was not consulted by any of them.

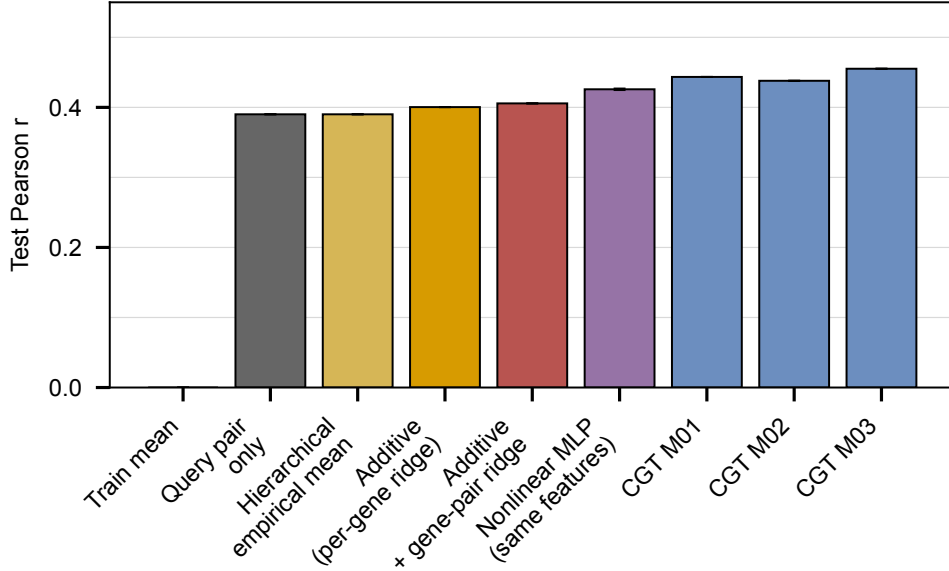


Figure 1: Test Pearson for every model. Error bars on the nonlinear baseline are the standard deviation across three seeds; the transformer bars are three separate training runs shown individually.

Capacity, not the graphs, closes most of the remaining gap. B5 sees exactly B1’s features, has no graph and no attention, and reaches 0.426. That closes about half the distance from B1 to the transformer. On mean squared error B5’s 0.003382 is better than two of the three checkpoints. Since Section 3.3 establishes that the nine graphs do not enter the forward pass, none of the B5-to-transformer difference can be attributed to them; what is left is the richer set aggregation of Eq. 4 and roughly eight times the parameters.

B4 is the result that changes what the metric means. B4 knows only which of 420 recurring gene pairs a triple contains. It never looks at the third gene. It reaches 0.390 test Pearson, 97 percent of B1’s and 86 percent of the best checkpoint’s. Those 420 pairs are the Kuzmin query doubles, the most frequent appearing 3,308 times, so B4 is close to a per-screen offset.

On a split that is random over records, then, most of what every model in Table 4 is scoring is the identity of the screen a triple came from. That is not trigenic biology and it would not survive a query-pair-disjoint split. It also explains why the additive null does so well on a label that was constructed to have the additive part removed: what B1’s per-gene coefficients absorb is largely query-pair structure reaching them through the genes that make up the query pairs.

Spearman compresses the ladder further. The rank correlations run 0.406 for B1, 0.412 for B2 and 0.417 to 0.421 for B5, against 0.424 to 0.426 for the transformer. The gap from the additive null to the transformer is 0.020 in Spearman against 0.045 in Pearson. Ranking is what a design-space selection consumes, so the smaller of the two is the more decision-relevant number.

6.2 Verifying the architecture reading ✗

Section 3 claims the encoder is strain-independent and the nine graphs do not enter the forward pass. Both are checkable rather than merely readable, and checking them also produces the per-record predictions that Section 8 lists as missing.

The stock evaluation path cannot run: the 010 LMDB stores `perturbation_type`: "deletion", which the current schema rejects. The route around it follows from the architecture. If the encoder really does not see the strain, then scoring a record needs only the perturbed gene indices, and the index space is the sorted base-graph node list, which is rebuildable from the genome and the build’s `gene_set.json` without touching the LMDB. `score_010_checkpoints_directly.py` does that, loads each checkpoint into the model class, and recomputes the metrics.

The check is that the recomputed numbers must reproduce what the original evaluation runs recorded. A wrong gene ordering or a mis-loaded weight would give a plausible but wrong number, so agreement to several digits is the evidence that the reconstruction is faithful.

Checkpoint	Val Pearson		Val MSE	
	recomputed	recorded	recomputed	recorded
M01 1zs9pcj3	0.451970	0.451963	0.003244	0.003244
M02 yv4r30bi	0.447319	0.447155	0.003258	0.003259
M03 c7671wgj	0.461917	0.461881	0.003163	0.003163

Table 5: Scoring the checkpoints through a hand-built forward pass that computes the encoder once and feeds it only perturbed-gene indices reproduces the recorded metrics. The residual difference is at the fourth decimal and is consistent with the original runs using mixed bf16 precision.

Two details had to be right for this to work, and both are worth recording because getting either wrong produced a plausible but wrong answer. The index space is the sorted genome gene set, 6,607 genes, not the build’s own 6,579 gene set; using the latter gave a validation Pearson of -0.001 to 0.002 . And the readout pools by *mean*, whereas the current class defaults to *sum*; using the default gave 0.420 to 0.438, close enough to look credible and wrong.

That reproduction is the direct verification of Section 3.2. The encoder was evaluated exactly once, on a strain-independent input, and the recorded per-split metrics came back. If the encoder had needed the strain, this could not have happened.

The graph ablation gives the other claim. Setting the graph penalty weight from 1.0 to 0.0 changes the maximum absolute test prediction by 9.0×10^{-7} , 1.2×10^{-6} and 1.3×10^{-6} for the three checkpoints, against a prediction spread of about 0.036. The nine graphs do not enter the forward pass.

6.3 Record-by-record agreement ✗

With per-record predictions available, the comparison can be made the way the DeWitt preprint makes it, between the predictions themselves rather than between their summary statistics.

Model	Pearson with		
	CGT M01	CGT M02	CGT M03
B1 additive ridge	0.752	0.726	0.759
B2 additive plus pair	0.759	0.734	0.765
B4 query pair only	0.729	0.706	0.735
B5 embedding MLP	0.788	0.784	0.810
CGT M01		0.788	0.821
CGT M02	0.788		0.804

Table 6: Correlation between models’ test-set predictions, 37,673 records. The transformer does not reproduce the additive model, which is where 010 departs from the MULTI-evolve case. But two transformer training runs agree with each other at 0.788 to 0.821, barely more than a transformer agrees with the additive model at 0.726 to 0.759, and no more than it agrees with the nonlinear baseline at 0.784 to 0.810.

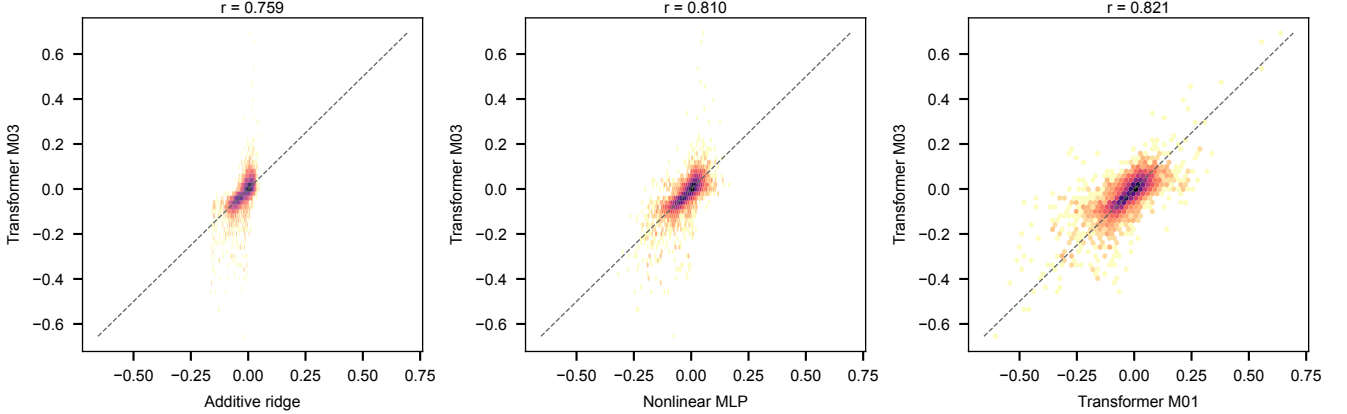


Figure 2: Test-set predictions of one model against another. The transformer is no closer to a second training run of itself than it is to a nonlinear baseline that has no graph and an eighth of the parameters.

The headline distinction from the preprint. DeWitt’s networks correlated with the additive model at $r > 0.999$, so the network *was* the additive model. Here the correlation is 0.73 to 0.76, so the 010 transformer is genuinely computing something the additive model does not. That conclusion should be stated plainly, because it is the opposite of the preprint’s finding and it is what the null was fit to test.

What that extra content is worth is a separate question. Seed-to-seed correlation between two transformer runs is 0.79 to 0.82. The part of the transformer’s output that is not additive is therefore largely not reproducible across training runs: a checkpoint disagrees with the additive model about as much as it disagrees with another checkpoint of the same architecture on the same split.

The models are wrong in the same places. Correlating the residuals $y - \hat{y}$ of B1 and each checkpoint gives 0.907, 0.897 and 0.918. Almost all of the error is shared, so the transformer is not fixing a class of records the additive model gets wrong; it is making a modest, broadly distributed improvement.

The advantage is real and bounded. A paired bootstrap over test records, 2,000 resamples, puts the Pearson gain over B1 at +0.043 with a 95 percent interval [+0.023, +0.063] for M01, +0.038 with [+0.019, +0.058] for M02, and +0.055 with [+0.034, +0.074] for M03. All three intervals exclude zero. B5 gains +0.025 with [+0.017, +0.033]. So the transformer beats a no-interaction null by a margin that is statistically established and is between two and six hundredths of a correlation.

Rankings diverge where selection happens. Among the 100 records each model calls most negative, B1 and M03 share 31, B5 and M03 share 45, and B1 and B5 share 46. At $K = 10,000$ all pairs exceed 0.91. The models agree about the bulk and disagree about the tail, which is the part a design-space selection reads.

Prediction spread. Test prediction standard deviations are 0.025 for B1, 0.029 for B5 and 0.036 to 0.038 for the checkpoints, against a test label standard deviation of 0.064. Every model is heavily shrunk toward the mean, as a squared-error fit on a noisy target should be.

7 The design space, and a defect found there ✗

Figure 1 of the DeWitt preprint is a scatter of network predictions against additive predictions across the whole combinatorial space, and its force comes from the near-identity of the two rankings: the variants selected for construction were recoverable without the network. The same comparison on 010 gave a result that turned out not to be about additivity at all.

7.1 The comparison as run ✗

Checkpoint `c7671wgj` scored three separate 010 inference spaces:

- `inference_1`, 4,370,595 triples over a 526-gene panel,
- `inference_2`, 479,195 triples,
- `inference_3`, 465,735,532 triples genome-wide, which is the space the panel-12 and panel-24 selections consumed.

Scoring the same triples with B1 and streaming the 4 GB prediction file gives, over the 325,631,845 triples of `inference_3` whose three genes all carry an additive coefficient, a Pearson correlation of 0.0018 and a top-100 overlap of zero. Stratifying by the training support of each triple’s least-observed gene does not account for it: even where all three genes have 800 or more training observations, $n = 867,669$, the correlation is 0.036.

On `inference_1` the same comparison behaves completely differently. B1 against the three checkpoints gives 0.548, 0.579 and 0.511.

7.2 One checkpoint disagreeing with itself ✗

The model is a deterministic function of the perturbed gene set, so a triple scored twice by one checkpoint must receive the same number twice. It does not.

Comparison	n	Pearson
Same triple, <code>inference_2</code> vs <code>inference_3</code>	2,404 triples	1.0000
Same triple, <code>inference_1</code> vs <code>inference_3</code>	330 triples	0.295
Per-gene mean, <code>inference_2</code> vs <code>inference_3</code>	1,274 genes	0.800
Per-gene mean, <code>inference_1</code> vs <code>inference_3</code>	395 genes	0.037
Per-gene mean, <code>inference_1</code> vs <code>inference_2</code>	179 genes	0.167

Table 7: Agreement of checkpoint `c7671wgj` with itself across inference runs. Runs 2 and 3 are identical on shared triples, mean absolute difference 0.000000, so they share a pipeline. Run 1 is unrelated to them. Per-gene means average over hundreds of triples each, so sampling noise cannot explain 0.037.

Deciding which run handles gene identity correctly needs a reference outside the transformer. The fitted coefficient vector $\hat{\beta}$ of Eq. 8 is one: it is estimated from measured training labels, keyed on systematic gene name, and independently validated at 0.400 test Pearson. If a run’s predictions carry real gene identity, a gene’s mean predicted effect should track $\hat{\beta}_g$.

Run	Own gene set	167 common genes
<code>inference_1</code>	0.579 (383 genes)	0.576
<code>inference_2</code>	0.048 (1,138 genes)	0.231
<code>inference_3</code>	-0.008 (3,180 genes)	0.174

Table 8: Pearson correlation between a run’s per-gene mean prediction and the additive ridge coefficient for that gene. The right column restricts to the 167 genes all three runs score with at least 200 observations, so the different gene coverage of the three runs cannot explain the gap.

Restricted to identical genes, `inference_1` tracks gene identity more than three times as strongly as the two runs that produced the selection.

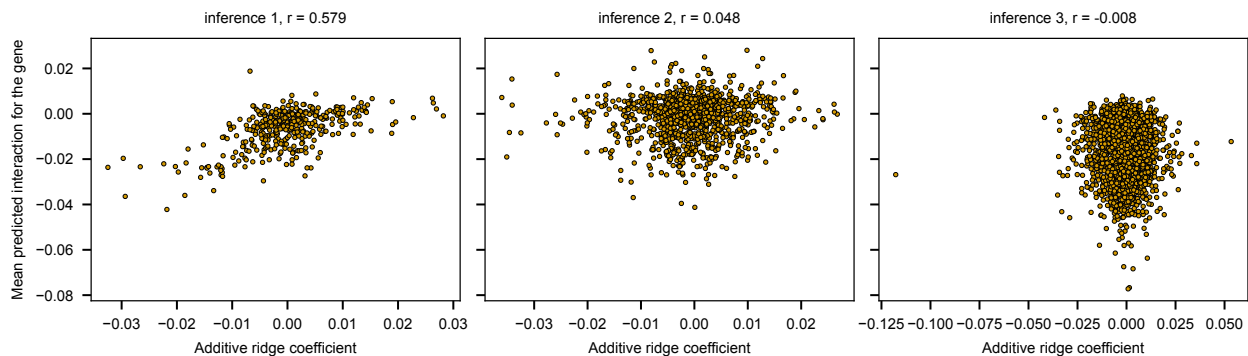


Figure 3: Per-gene mean prediction against the additive ridge coefficient, for each of the three inference runs of checkpoint `c7671wgj`. The additive coefficient is fit on measured training data and keyed on systematic gene name, so it anchors gene identity from outside the transformer.

7.3 What is and is not established ✗

Established: the three runs do not agree; runs 2 and 3 are byte-identical on shared triples; run 1 tracks an external gene-identity anchor and runs 2 and 3 largely do not; the selection consumed run 3.

Hypothesis (untested). A gene index or embedding misalignment in the shared `inference_2` and `inference_3` dataset build would produce this signature, predictions that are self-consistent and stable yet detached from gene identity. The inference builds carry no `gene_set.json` of their own while the training build has one of 6,579 genes, so the index used at inference time is reconstructed rather than loaded. No cause has been confirmed and no fix has been attempted.

7.4 Seed instability in the selection tail, separately ✗

On `inference_1`, where predictions do track gene identity, the three checkpoints still disagree substantially about which triples are extreme. Checkpoint-to-checkpoint Pearson is 0.524 to 0.560, no higher than each checkpoint's 0.511 to 0.579 agreement with the additive model, and top-100 overlap between two checkpoints is 0.21 to 0.35. Two independent training runs of one architecture on one split nominate largely different top-100 triples, so a selection taken from a single checkpoint's tail is not reproducible across seeds even where the inference pipeline is sound.

8 What this establishes, and what it does not ✗

8.1 Established ✗

The additive null had never been fit on 010, and now it has, on the same records, split and label the transformer used, with the ridge penalty chosen on validation and reported on test.

Every model in the comparison receives the same information, the identity of the three perturbed genes. That is a reading of the code rather than an assumption: the encoder is applied at batch size one to a strain-independent input, and the nine interaction graphs appear only in a loss term.

The transformer's margin over a model that cannot represent interaction is real and small. The interesting part of the ladder is not the top of it but the bottom: a model that never looks at the third gene is already most of the way there, which says the split is measuring screen structure at least as much as trigenic biology.

8.2 Not established ✗

The reproduction is faithful but not bit-exact. The hand-built forward pass matches the recorded metrics to the fourth decimal, not exactly. The original runs used mixed bf16 precision and this one runs in fp32, which is the likely source, but that has not been verified by rerunning in bf16. Conclusions in Section 6.3 rest on correlations at the second decimal and are not sensitive to a fourth-decimal shift.

One seed of the nonlinear baseline enters the paired analysis. Table 6 uses B5 seed 0. The three seeds agree closely on aggregate metrics, but their record-level agreement with each other was not measured, so the B5 column should be read as one draw.

The trigenic biology question is not answered. B4 shows that a per-query-pair offset reaches most of the metric on a split that is random over records. Nothing here separates trigenic biology from screen structure, because no split was run that would. The experiment that would is a query-pair-disjoint split, in which no query double appears in both training and test, and it has not been run.

The design-space defect has no confirmed cause. Section 7 establishes that one checkpoint’s inference runs disagree and identifies which of them tracks an external gene-identity anchor. It does not identify why, and the index-misalignment reading is labeled a hypothesis there because it has not been tested.

Nothing here bears on the architecture’s merits elsewhere. The reduction in Section 3.4 is specific to the 010 configuration, where the masking and propagation arms are off and the perturbation transform is one block. A configuration that switches those on would have a strain-dependent encoder path and would need its own reading.

8.3 Next ✗

1. Measure B5 seed-to-seed prediction correlation, so the transformer’s 0.79 to 0.82 seed agreement in Table 6 has a like-for-like reference rather than being read against the additive model alone.
2. Run a query-pair-disjoint split and refit every row of Table 4. This is the measurement that would separate trigenic biology from screen batch, and B4’s value is what makes it necessary.
3. Test the index-misalignment hypothesis by scoring one small triple set through both inference build paths and comparing against `inference_1`. If it holds, the panel-12 and panel-24 selections need regenerating; the affected prediction file is pinned by sha256 806ef044...3550 in `results/prediction_calibration_stats.csv`.
4. Report a seed-averaged or ensembled ranking for any future selection rather than one checkpoint’s tail, given the 0.21 to 0.35 top-100 overlap between checkpoints.
5. Migrate the 010 build’s `perturbation_type` values onto the current schema vocabulary, or record explicitly that the build is frozen at a pre-refactor schema. Until then the stock evaluation path cannot run against it, which is what made the per-record comparison hard to obtain.